Wykład 1 - Podstawy

Wirtualizacja

- **wirtualizacja** - tworzenie abstrakcji, czyli warstwy pośredniej, sprawiającej, że zasoby fizyczny wyglądają inaczej niż w rzeczywistości
- trzy etapy rozwoju wirtualizacji:
  - jeden serwer, wiele usług
    - niski koszt sprzętu
    - prosta architektura
    - rywalizacja o zasoby pomiędzy usługami
    - niezgodność bibliotek
    - podatność na awarię (jedna usługa może wywalić cały serwer)
  - jeden serwer, jedna usługa
    - izolacja usług
    - brak konfliktów
    - marnotrastwo mocy obliczeniowej
    - wysokie koszty sprzętu, energii, miejsca
    - trudne zarządzanie
  - maszyny wirtualne
    - izolacja usług
    - brak konfliktów
    - łatwe zarządzanie
    - precyzyjne wydzielanie zasobów
    - lepsze wykorzystanie fizycznego sprzętu
- **klaster wirtualizacji** - wiele serwerów łączy się w klaster, gdy jedno VM nie wystarczy
  - wysoka skalowalność i dostępność
  - niższe koszty operacyjne (w trakcie działania)
  - złożone
  - wysoki koszt początkowy (jednorazowy przy zakupie sprzętu)

Hypervisor (VMM)

- **hypervisor (VMM)** - warstwa oprogramowania pomiędzy sprzętem a maszynami wirtualnymi
- trzy zadania hypervisora:
  1. zapewnić środowisko identyczne z fizycznym (wirtualka ma nie wiedzieć, że jest wirtualką)
  2. zrobić to minimalnym kosztem wydajnościowym
  3. zachować kontrolę nad fizycznymi zasobami
- **narzut wirtualizacji** - zużycie zasobów przez hypervisora

Wykład 2 - Sposoby wirtualizacji

Rodzaje wirtualizacji

(od najwolniejszej do najszybszej):

1. **Pełna emulacja**
   - wirtualizator przejmuje wszystkie wywołania
   - odwzorowanie sprzętu czysto programowo
   - może udawać zupełnie inny sprzęt niż fizyczny
   - bardzo wolne ale daje największe możliwości
2. **Pełna wirtualizacja**
   - wirtualizator śledzi wywołania i w miarę możliwości udostępnia zasoby fizyczne bezpośrednio

    - tylko problematyczne wywołania są emulowane programowo
    - nie wymagają modyfikacji systemu operacyjnego gościa
    - np. VirtualBox
3. **Wirtualizacja sprzętowa**
    - pełna wirtualizacja wspierana przez procesor
    - procesor ma specjalne instrukcje do obsługiwania uprzywilejowaych instrukcji
    - dzisiaj standard
4. **Parawirtualizacja**
    - modyfikacja jądra systemu gościa
    - system wie że jest wirtualizowany, więc używa specjalnego API wirtualizatora zamiast normalnych wywołań sprzętowych
    - dużo szybsze
5. **Wykorzystanie wspólnego jądra**
    - kilka instancji tego samego systemu współdzieli jądro
    - wyparte przez konteneryzację
6. **Konteneryzacja**
    - wspólne jądro i izolacja procesów i zasobów w kontenerze
    - znacznie lżejsze niż pełna wirtualizacja
    - zdecydowanie słabsza izolacja

Poziomy uprzywilejowania Intel

- 4 poziomy (ringi):
    - ring 0 - jądro systemu operacyjnego - pełne uprawnienia
    - ring 3 - aplikacje użytkownika - minimalne uprawnienia
    - ringi 1 i 2 są historycznie dla sterowników, dzisiaj nieużywane
- idea polega na tym że niektóre instrukcje procesora lub dostęp do niektórych rejestrów są dostępne tylko z ringu 0
    - inaczej dostajemy *General Protection Fault*
- jeżeli VMM działa w ringu 0, to system gościa musi być przesunięty na ring 1 lub 3 (model 0/1/3 lub 0/3/3), co powoduje problemy:
    - **ring aliasing** - system myśli że jest w ring 0, a tak naprawdę jest niżej uprzywilejowany
    - każda uprzywilejowana instrukcja generuje błąd, który musi być obsłużony przez VMM - bardzo wolne i potencjalnie niebezpieczne
- rozwiązanie problemu - Intel VT-x
    - dwa tryby pracy procesora - **VMX (Non-)Root** tryby odpowiednio dla systemów gości/VMM
    - zarówno VMM jak i system gościa działają w ringu 0
    - gdy gość chce wykonać uprzywilejowaną operację, procesor przekazuje sterowanie automatycznie do VMM - **VM Exit**
    - po obsłużeniu sytuacji przez VMM - **VM Entry**

Dwa typy hypervisora

- **typ 1**
    - *bare metal*
    - działa bezpośrednio na maszynie fizycznej
    - wirtualizator zarządza np. sieciami, przydziela adresy wirtualkom
    - wydajniejszy i bezpieczniejszy
    - np. VMware
- **typ 2**
    - na maszynie fizycznej działa OS, w którym działa zarządca

- mamy opcję stosunkowo łatwej migracji z jednego gospodarza na innego
- należy dbać o bezpieczeństwo obu elementów
- mniej wydajny ale łatwiejszy do doinstalowania
- np. VirtualBox

Wykład 3 - Podstawowe pojęcia, sprzęt

Podstawowe pojęcia

- **obraz (*image*)** - plik zawierający dane maszyny (istnieje nawet gdy VM nie działa)
- **instancja** - uruchomiona maszyna z przydzielonymi zasobami (istnieje tylko gdy VM działa)
- **definicja** - plik konfiguracyjny opisujący parametry maszyny bez przydzielonych zasobów
- **klonowanie**
  - tworzenie niezależnej kopii maszyny
  - obie VM działają niezależnie od siebie
  - zwykle konieczne jest zmodyfikowanie konfiguracji (nowy adres IP, nazwa hosta)
  - stosowane do skalowania usług
  - **nie używać do backupów** - kopia zwykle przechowywana w tym samym miejscu co oryginał
- **szablon**
  - wzorzec do tworzenia nowych VM
  - zawiera system operacyjny, aplikacje, konfigurację
  - wymaga "dokończenia" konfiguracji - adres IP, nazwa itp.
  - przydatny w automatyzacji (np. Ansible)
- **migawka (*snapshot*)**
  - zapisanie stanu maszyny w konkretnym momencie
  - można tworzyć wiele migawek
  - dane przechowywane przyrostowo (różnice, nie pełna kopia)
  - można przywracać stan do dowolnej migawki
  - nie zastępują kopii zapasowych - migawki generują narzut i są zależne od obrazu bazowego

Przypomnienie z ISO/OSI

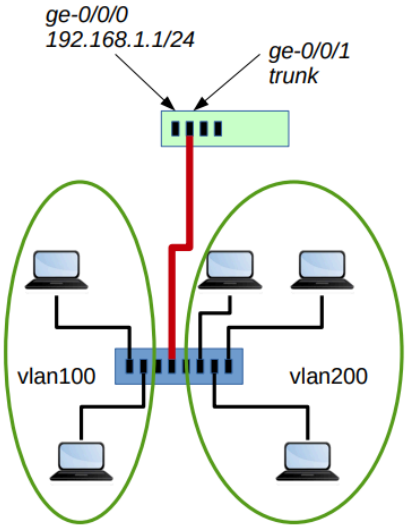
- **Hub** - koncentrator (warstwa 1 - fizyczna)
  - wzmacniacz sygnału z wieloma portami
  - wszystkie urządzenia podłączone do huba tworzą **domenę kolizyjną**
    - tylko jedno urządzenie może nadawać w danym momencie
  - dziś rywalizacja o łącze rozwiązana, huby wyszły z użycia
  - jedna domena rozgłoszeniowa
- **Bridge** - most (warstwa 2 - łącza danych)
  - łączy dwa segmenty sieci i rozpoznaje adresy MAC
  - kluczowa cecha - separuje domeny kolizyjne
  - obie strony mostu należą do jednej **domeny rozgłoszeniowej** - obszaru sieci, w którym urządzenia mogą komunikować się wyłącznie na podstawie adresów MAC
- **Switch** - przełącznik (warstwa 2 - łącza danych)
  - **multiport bridge** - most z wieloma portami
  - uczy się adresów MAC i wysyła ruch tylko na właściwy port
  - cały switch - jedna domena rozgłoszeniowa
- **Router** - ruter (warstwa 3 - sieciowa)
  - działa na podstawie adresów IP, nie MAC
  - przekazywanie pakietów pomiędzy sieciami
  - rozdziela domeny rozgłoszeniowe (każdy port - osobna domena rozgłoszeniowa)

Wykład 4 - VLAN, VirtualBox

VLAN

- mechanizm logicznego podziału jednego fizycznego przełącznika na kilka niezależnych sieci
- każdy VLAN tworzy osobną domenę rozgłoszeniową (jakby oddzielny switch)
- w praktyce: jeden VLAN - jedna podsieć IP
- komunikacja między VLANami wymaga routera lub firewalla
- rodzaje portów w terminologii Cisco:
  - **Access Port** - port bez znakowania - urządzenie podłączone do tego portu nie wie, że jest w VLANie
  - **Trunk Port** - port ze znakowaniem 802.1q. - typowo łączy switch z routerem/firewallem, przesyła ramki z kilku VLANów jednocześnie, każda z tagiem identyfikującym przynależność
    - gdy firewall jest podłączony *trunk portem*, cały ruch między VLANami przechodzi przez ten port dwukrotnie - wąskie gardło

Przykładowa konfiguracja firewalla




```
interfaces {
    ge-0/0/0 {
        description "management";
        unit 0 {
            family inet {
                address 192.168.1.1/24;
    } } }
    ge-0/0/1 {
        description "to cluster";
        unit 0 {
            family ethernet-switching {
                port-mode trunk;
                vlan {
                    members [ VLAN_DEV VLAN_ACC ];
    } } } }
```

- jeden firewall z dwoma interfejsami
- `ge-0/0/0` - port zarządzania
- `ge-0/0/1` - *trunk port* łączący switch z dwoma sieciami - `vlan100` i `vlan200`
- warstwowa konfiguracja:
  - **interfejsy fizyczne**: *trunk port* z listą VLANów
  - **interfejsy logiczne VLAN** - każdy dostaje adres IP (bramę domyślną dla danej podsieci)
  - **definicje VLANów** - przypisanie ID i interfejsu logicznego
  - **strefy bezpieczeństwa**
  - **polityki** - reguły przepuszczania ruchu pomiędzy strefami

Wirtualny przełącznik

- kluczowe pojęcie dla wirtualizacji
- działa tak samo jak fizyczny przełącznik, ale w softwarze
- łączy interfejsy sieciowe VM z interfejsami fizycznymi hosta
- obsługuje VLANy i tablice MAC

Oracle VirtualBox

- popularny **wirtualizator typu 2**
- darmowy do użytku osobistego
- tryby zapisu a migawki:

Tryb	Migawki	Współdzielenie	Opis	Zastosowanie
Normal	Tak	Nie	Domyślny, tworzy plik różnicowy, możliwe cofnięcie zmian	Zwykłe użycie
Write-through	Nie	Nie	Dane zapisywane bezpośrednio do obrazu bazowego, migawki niedostępne	Dla dysków w których dane mają przetrwać *revert* VM
Shareable	Nie	Tak (r/w)	Wspólne dla wielu maszyn, fixed-size, własna synchronizacja	Wspólne dane do zapisu
Immutable	Wyjątkowo, w tym trybie tworznie migawek jest bez sensu, bo i tak chcemy startować od stanu bazowego	Tak (r/o)	Dane tracone po wyłączeniu VM - maszyny startują zawsze od tego samego stanu	Np. dla środowisk testowych które mają być za każdym razem przywracane do stanu początkowego
Multiattach	Tak	Tak	Każda VM otrzymuje własny plik różnicowy od obrazu bazowego, każda VM widzi tylko własne zmiany	Grupy maszyn z takim samym systemem bazowym, ale pracujące niezależnie od siebie
Read-Only	Nie	Tak	Dysk tylko do odczytu, tak jak płyty CD/DVD	Instalatory, bazy danych tylko do odczytu, zabezpieczenie przed modyfikacją ważnych danych

- działanie migawek:
  - każda migawka jest plikiem różnicowym względem poprzedniego
  - przy odczycie VirtualBox czyta od najnowszego pliku do obrazu bazowego
  - zapis zawsze do najnowszego pliku różnicowego
  - klonowanie bieżącego stanu konsoliduje wszystkie miigawki w jeden obraz
- *promiscuous mode* - tryb nasłuchu wszystkich ramek, przydatne do sniffowania
- tryby sieciowe:
  - **NAT**

    - wirtualizator sam tłumaczy adresy
    - adresy dostaje przez DHCP
    - VM może się łączyć z gospodarzem, ale gospodarz nie może się połączyć z VM
    - możliwość udostępnienia usługi z VM do gospodarza poprzez *port-forwarding*
  - **NAT Network**
    - to samo co NAT, ale maszyny wirtualne widzą się nawzajem i mogą się komunikować ze sobą
  - **Bridged Networking**
    - VM jest podłączona do wirtualnego przełącznika który korzysta z fizycznego interfejsu gospodarza
    - VM dostaje prawdziwy adres w sieci lokalnej
    - komunikacja VM z hostem tak jakby komunikować się z innym urządzeniem w sieci - zależy od firewalla
  - **Internal Networking**
    - VM widzą tylko siebie nawzajem
    - zupełna izolacja od gospodarza
  - **Host-only Networking**
    - VM komunikują się i ze sobą, i z gospodarzem
    - VM nie mają dostępu do internetu
- podsumowanie trybów sieciowych:

Tryb	VM do hosta	Host do VM	VM i VM	VM do LAN	LAN do VM
NAT	Tak	*port-forwarding*	Nie	Tak	*port-forwarding*
NAT Network	Tak	*port-forwarding*	Tak	Tak	*port-forwarding*
Bridged	Zależy od sieci, nie od VB	Zależy od sieci, nie od VB	Tak	Tak	Tak
Internal	Nie	Nie	Tak	Nie	Nie
Host-Only	Tak	Tak	Tak	Nie	Nie

Wykład 5 - KVM

Architektura KVM

- **hypervisor typu 1**, chociaż niektórzy się kłócą
- trzy warstwy:
  1. *kvm.ko* - moduł jądra linuxa zapewniający wirtualizację sprzętową
  2. *QEMU* - emulator urządzeń, działający w przestrzeni użytkownika
  3. *libvirt* - demon zarządzający maszynami, umożliwia zarządzanie całym klastrem VM
- przez to że QEMU działa w przestrzeni użytkownika, zwykłe narzędzia systemowe widzą maszyny wirtualne:
  - `ps` widzi procesy VM
  - `tcpdump` - umożliwia podsłuchiwanie ruchu sieciowego na interfejsach
  - `iptables` - umożliwia filtrację ruchu

Formaty obrazów dysków KVM

- **raw**
  - prosty, szybki
  - zajmuje od razu pełną zadeklarowaną przestrzeń na dysku gospodarza
  - brak migawek, kompresji
- **qcow2**

  - preferowany
  - zajmuje tylko tyle miejsca, ile zużywa VM
  - obsługuje migawki
  - możliwa kompresja i szyfrowanie
  - tryby wstępnej alokacji:
    - *metadata* - alokuje tylko metadane, dane zapisywane dynamicznie
    - *falloc* - rezerwuje miejsce przez syscall
    - *full* - alokuje od razu całą przestrzeń (tak jak *raw*)

Migawki KVM

- przechowują:
  - stan obrazu dysku
  - stan pamięci RAM
- maszyna **musi być wstrzymana** do tworzenia migawki
- *revert* przywraca dysk i pamięć do stanu z czasu migawki

Pamięć masowa KVM

- zasoby dyskowe zarządzane przez **pule** (ang. *pools*)
- typy pul:
  - **dir** - katalog z plikami obrazów (najpopularniejszy)
  - **disk** - fizyczny dysk z automatycznie tworzonymi partycjami
  - **partition** - sformatowana partycja
  - **nfs/gluster** - sieciowy system plików
  - **iscsi/scsi** - wolumeny muszą być utworzone wcześniej
  - **lvm** - wolumenty zakładane automatycznie w grupie logicznej

Tworzenie i klonowanie maszyny KVM

- tworzenie obrazu dysku: polecenie `qemu-img create`
- instalacja maszyny: `virt-install`
- migawka bazowa po konfiguracji: `virsh snapshot-create-as`
- klonowanie: `virt-clone`

Sieci

- domyślnie mamy sieć **default** z mostem **virbr0**
- interfejsy VM są widoczne w systemie hosta (polecenie `ip a`)
- opcje konfiguracji sieci:
  - **własny most (briged)** - np. w pliku `/etc/network/interfaces`
  - **sieć KVM przez XML** - np. polecenie `virsh net-define br10.xml`
- zasady bezpieczeństwa sieci VM - należy zapewnić:
  - izolację VM od siebie - nie wszystkie VM muszą widzieć siebie nawzajem
  - izolację VM od hosta
  - izolację VM od internetu - np. serwery baz danych nie powinny mieć dostępu na zewnątrz

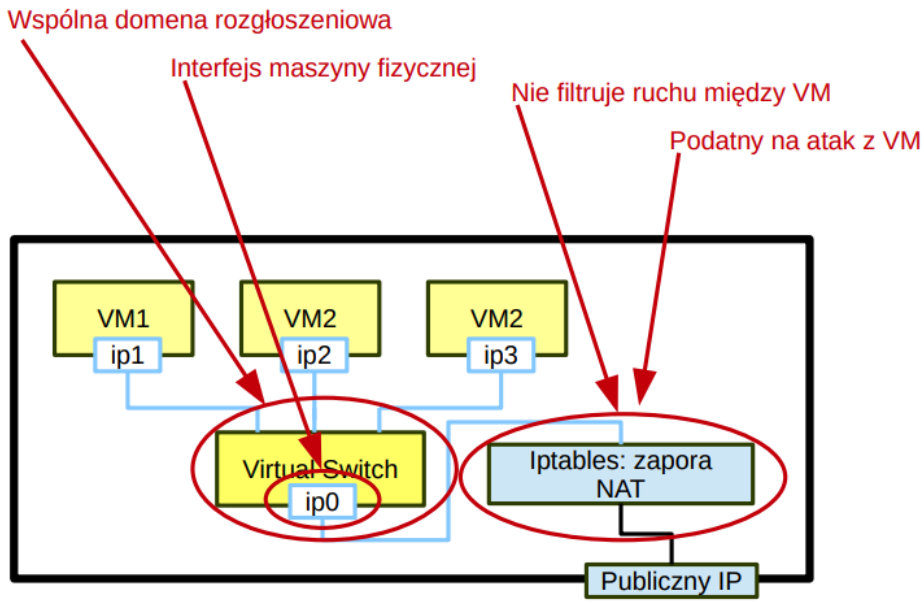
Wykład 6 - Bezpieczeństwo sieci

Gdzie zabezpieczać VM - trzy warstwy

- **Firewall VM** - np. `iptables` wewnątrz VM - ochrona wewnętrzna maszyny wirtualnej i tylko ją chroni

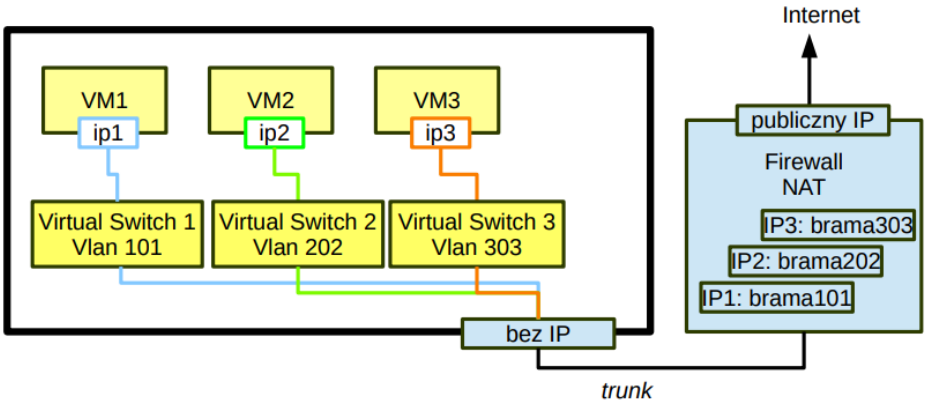
- **Firewall hosta** - np. `iptables` na hoście - działanie na hypervisorze, może filtrować ruch wchodzący i wychodzący ze wszystkich VM, ale wymaga dobrej konfiguracji żeby nie przepuszczać niezauważenie ruchu pomiędzy VM
- **Firewall zewnętrzny** - fizyczne urządzenie przed całą infrastrukturą, chroni przed internetem, ale nie widzi ruchu pomiędzy VM na tym samym hoście

Przykład 1 - wspólna sieć



- VM -> VM - swobodny dostęp, bo przełącznik nic nie filtruje
- Host <-> VM - swobodny dostęp, przez interfejs `ip0` na przełączniku
- przejęcie jednej wirtualki daje atakującemu dostęp do wszystkich pozostałych VM i do hosta

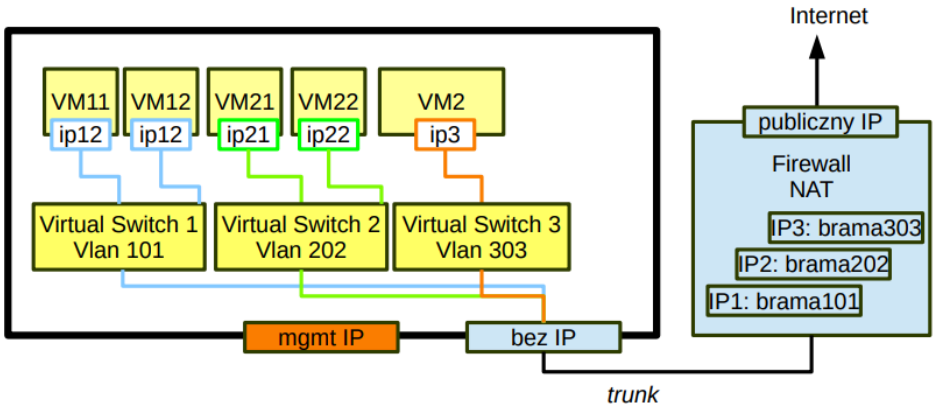
Przykład 2 - paranoja



- każda VM ma osobny VLAN, osobny wirtualny przełącznik i cały ruch przechodzi przez centralny firewall
- VM <-> VM - ruch izolowany przez domeny rozgłoszeniowe
- Host <-> VM - brak komunikacji, bo host nie posiada interfejsu w żadnym VLANie VM
- przejęcie VM nie oznacza przejęcia całej architektury
- podejście paranoiczne powoduje problemy:

- przy dużej komunikacji pomiędzy usługami firewall staje się wąskim gardłem (cały ruch musi przez niego przejść)
- utrudniona administracja, konieczność osobnej sieci zarządczej

Przykład 3 - kompromis, grupowanie maszyn



- grupowanie maszyn ze względu na ich stopień zagrożenia/ekspozycji (np. serwery publiczne vs. wewnętrzne)
- grupowanie ze względu na przeznaczenie i stopnia współpracy (np. maszyny jednej aplikacji w jednej grupie)
- maszyny w jednej grupie komuynikują się przez wspólny przełącznik (bez firewalla)

Sieć zarządcza

- oddzielna sieć służąca wyłącznie do administracji infrastrukturą
- na poziomie hosta służy do tworzenia/usuwania VM, aktualizacji, obsługi awarii
- na poziomie VM służy do konfigurowania systemu i aplikacji, ale też do obsługi awarii
- ograniczenia firewalla w sieci zarządczej:
  - firewall nie przeprowadza autoryzacji
  - firewall nie filtruje ruchu między VM na tym samym przełączniku zarządczym

Host bastionowy

- dodatkowy serwer pośredniczący w dostępie administracyjnym
- zapewnia autoryzację i logi systemu
- dodaje złożoność ale poprawia audytowalność

Porady dotyczące bezpieczeństwa sieci

- prosta topologia - im prostsza tym tańsza i łatwiejsza w obsłudze
- kiedy stosować osobną sieć zarządczą - gdy sieć robocza może być przeciążona
- sieć zarządcza nie powinna mieć dostępu do internetu, lepiej postawić wewnętrzne usługi
- dostęp do sieci zarządczej z internetu powinien przechodzić przez VPN lub bastion
- VLAN hosta powinien być inny niż VLANy VM
- przełączniki w sieci zarządczej mogą być tańsze, bo ruch zarządczy jest niewielki

CloudSim

- narzędzie do symulowania zachowania klastrów i chmur obliczeniowych bez fizycznej infrastruktury
- główne klasy:
  - **Datacenter** - reprezentuje cały klaster, zawiera parametry centrum danych (architektura, OS, koszt), jak i polityki przydziału VM do hostów

  - **Host** - fizyczna masyzna w klastrze, definiuje zarządzanie pamięcią RAM, przepustowością, CPU, podziałem CPU między VM
  - **VM** - maszyna wirtualna, zawiera przydział mocy obliczeniowej między zadania VM
  - **DatacenterBroker** - pośrednik przydzielający zasoby
  - **Cloudlet** - zadanie obliczeniowe, definiowane przez:
    - długość zadania
    - rozmiar danych wejściowych i wyjściowych
    - jak zadanie wykorzystuje zasoby
- przepływ symulacji:
  1. stwórz Datacenter z liczbą fizycznych hostów
  2. stwórz brokera
  3. stwórz VM i przypisz do brokera
  4. stwórz Cloudlety i przypisz do brokera
  5. Uruchom symulację i przeanalizuj wyniki

Wykład 7 - Zarządzanie energią w centrach danych

Przybliżenie problemu

- praktycznie cała energia elektryczna jest zamieniana w ciepło
- przegrzanie -> awaria sprzętu
- większy koszt - zarówno dla serwerów jak i chłodzenia
- emisja CO_2
- drogie chłodzenie (miejsce, sprzęt, do 50% poboru mocy na samo chłodzenie)

PUE - Power Usage Effectiveness

- najważniejszy wskaźnik efektywności energetycznej centrum danych:

$$\mathrm{PUE} = \frac{P_{\mathrm{total}}}{P_{\mathrm{IT}}}$$

  - P_{total} - całkowita moc pobierana przez centrum danych (serwery + chłodzenie + zasilanie + wszystko inne)
  - P_{IT} - moc pobierana wyłącznie przez sprzęt IT (serwery, switche itp.)
- idk czemu to nie jest na odwrót no ale dobra
- idealne PUE wynosi 1 (cała energia idzie do IT bez strat)

Narzędzia i metody redukcji zużycia energii

- **monitorowanie** - nie można zarządzać czymś o czym nic nie wiadomo
- **sterowanie poziomem zużycia** - np. ograniczenie częstotliwości CPU w godzinach małego ruchu
- **kolejkowanie i alokacja zadań** - skupienie obciążenia na mniejszej ilości serwerów, resztę usypiać
- **odzysk ciepła** - pompy ciepła, ogrzewanie budynków ciepłem z serwerowni
- **free-cooling** - np. chłodzenie powietrzem z zewnątrz gdy temperatura na to pozwala
- **korzystna lokalizacja** - np. Skandynawia, gdzie jest zimno i tańsza energia odnawialna
- **separacja traktów ciepłych i zimnych** - zapewnienie efektywnego przepływu zimnego i ciepłego powietrza w pomieszczeniu, zwiększając efektywność chłodzenia

Analiza zużycia energii

- należy szacować zarówno **moc maksymalną**, jak i **typową/średnią**

- np. jeżeli moc maksymalna serwerowni wynosi 19150 W, a średnia - 12320 W, to zużywamy średnio 64% mocy maksymalnej, co daje 36% przestrzeni do sterowania
- przy odpowiednim sterowaniu jest możliwe obniżenie faktycznego zużycia poniżej średniej - po to właśnie zarządza się energią

Monitorowanie zużycia energii

- **IPMI**
  - *Intelligent Platform Management Interface*
  - do zarządzania serwerem przez dedykowany kontroler, niezależnie od systemu operacyjnego, nawet gdy serwer jest wyłączony
  - można odczytać pobór mocy, temperaturę, stan wentylatorów
  - raczej rzadsze pomiary - zwykle co ok. 5 minut, minimalnie 15 sekund
  - odczyt zdalny trwa 1-2 sekundy, trochę wolno
- **SNMP** - do monitorowania urządzeń sieciowych (przełączniki, routery, firewall)
- **MSR + RAPL**
  - dla procesorów
  - **MSR** - *Model Specific Registers* - rejestry bezpośrednio w procesorze
  - **RAPL** - *Running Average Power Limit* - mechanizm do estymacji i ograniczenia poboru mocy CPU
  - bardzo częste odczyty - nawet 10-100 Hz
  - **estymacja, nie prawdziwy pomiar**

Wykład 8 - Kontynuacja o energii

ACPI

- *Advanced Configuration and Power Interface*
- zarządza konfiguracją urządzeń i poborem mocy poprzez hierarchię stanów:
  - **stany globalne G*** - cały komputer
    - **stany S*** - poziom uśpienia komputera
  - **stany procesora C*** - poziomy aktywności procesora
    - **stany P*** - stany wydajności procesora

DVFS

- *Dynamic Voltage and Frequency Scaling*
- model energetyczny bramki CMOS:

$$P = P_{\text{leak}} + P_{\text{short}} + P_{\text{dyn}}$$

  - P - moc pobierana przez procesor
  - P_{leak} - moc tracona w wyniku upływu w stanie ustalonym - gdy tranzystor jest wyłączony, mimo to przepływa przez niego mały prąd, proporcjonalny do napięcia U
  - P_{short} - moc tracona w wyniku upływu przez tranzystory w chwili przełączania - przy przełączaniu przez chwilę występuje zwarcie pomiędzy oboma tranzystorami - moc proporcjonalna do częstotliwości f
  - P_{dyn} - moc potrzebna do przełączania tranzystorów (**dominujące w działającym procesorze**)
- $$P_{\text{dyn}} = \alpha C U^2 f$$
  - α - intensywność przełączeń na takt
  - C - pojemność elektryczna
  - U - napięcie

- f - częstotliwość

Sterowanie stanami procesora w Linuxie

- wybór stanu C - *idle governor*
  - esymacja czasu bezczynności, timer events oriented, ladder, halt polling
- wybór stanu P - *frequency governor*
  - performance, powersave, ondemand, conservative

Algorytmy lokalne vs polityki globalne

	Lokalne	**Globalne**
Zakres	Jedno urządzenie	Cały klaster
Skala czasowa	Krótka	Długa
Wiedza o zadaniach	Brak	Tak (prognozy)

Konsolidacja zadań

- zmniejszenie ilości wykorzystywanych serwerów do wykonania zadań
- działa, ponieważ pobór mocy **nie jest** proporcjonalny do obciążenia
  - mamy próg włączenia urządzenia (bezczynne urządzenie tez pobiera prąd)
  - zużycie prądu wraz ze wzrostem obciążenia też nie rośnie liniowo

Algorytmy alokacji zadań

- **predykcyjne** - działają w szerszej skali, korzystają z historii obciążenia
- **reaktywne** - działają na bieżąco:
  - **BFD - Best Fit Decreasing** - sortowanie VM od najbardziej do najmniej zasobożernych
  - **PCA-BFD** - uwzględnia efektywność energetyczną serwera
  - **LPBP** - migracja VM z serwera o największym poborze mocy
- formalne zadanie optymalizacji - minimalizaja sumy

$$\min_{x_s} \left(\sum_{s=1}^{S_f} \sum_{k=1}^{K_s} P_s^k x_x^k \right)$$

  - minimalizacja sumy poboru mocy procesory razy wybór stanu procesora
  - problem ma charakter ILP - *Integer Linear Programming*
  - zmienne binarne powodują eksplozję wymiarowości zadania przy dużych klastrach

Wykład 9 - VMware ESXi

Architektura

- **wirtualizator typu 1**
- **VMKernel** - lekkie jądro systemu oparte na POSIX, dostosowane do obsługi wirtualizacji
- **VMM** - każda VM ma swoje VMM, ESXi używa rozproszonego hypervisora, jądro dostarcza niskopoziomowe wsparcie, a reszta logiki działa w przestrzeni użytkownika

Pamięć masowa

- **Datastore** - kontener na pliki VM (analogiczne do pul w KVM)
- na urządzeniach blokowych mamy system plików VMFS
  - może rozciągać się na wiele urządzeń - łatwa skalowalność
  - ma mechanizmy blokowania dla wielodostępu
  - obsługuje migawki

- typy połączeń storage:
  - **SCSI** - lokalnie, bezpośrednio do serwera
  - **iSCSI** - przez sieć LAN
  - **Fibre Channel (FC)** - przez specjalną kartę HBA (*Host-Bus-Adapter*), najwyższa separacja
  - **Fibre Channel over Ethernet (FCoE)** - karta CNA
- formaty dysków:
  - **Eager zeroed thick** - całe miejsce zarezerowane i wyzerowane od razu - najwyższa wydajność
  - **Lazy zeroed thick** - miejsce zarezerwowane, ale zerowane przy zapisie
  - **Thin provisioning** - zajmuje tylko tyle miejsca ile potrzebują VM - oszczędność miejsca, ale ryzyko wyczerpania
- iSCSI vs FC
  - FC - osobna sieć i przełączniki, pełna separacja ruchu, wysoki koszt
  - iSCSI - wspólna infrastruktura sieciowa, tańsza, można izolować VLANami

Architektura sieci ESXi

- cztery elementy:
  1. **porty fizyczne (*uplinks*)** - fizyczne karty sieciowe serwera
  2. **wirtualny przełącznik (*vSwitch*)** - łączy porty fizyczne z wirtualnymi portami VM
  3. **grupy portów** - logiczne grupy portów wirtualnych do których łączą się VM (każda grupa ma przypisany VLAN ID, co umożliwia izolację między VM)
  4. **wirtualne porty** - punkty podłączenia interfejsów sieciowych VM

Redundancja sieci

- aby uniknąć *single-point-of-failure*, stosuje się agregację łącz
- **HA passive-active** - jeden port aktywny, drugi czeka na awarię
- **LACP** - aktywna agregacja kilku kart w jedno logiczne złącze, zarówno redundancja, jak i równoważenie obciążenia, jeden port LACP na grupę portów

Wykład 10 - Klastry

Cele klastrowania

- skalowalność - więcej zasobów niż pojedyczy serwer
- możliwość efektywniejszego zarządzania posiadanymi zasobami
- niezawodność - przy awarii VM może być automatycznie zrestartowana na innym węźle

Problemy z klastrowaniem

- zarządzanie siecią - spójność konfiguracji i bezpieczeństwa
- zarządzanie pamięcią masową - wspólna lub rozproszona
- zarządzanie VM - tworzenie, klonowanie, migracja, migawki, kopie zapasowe
- zarządzanie monitoringiem - zasobów, stanami VM, logi

VMware vSphere

- kompletny zestaw narzędzi VMware do zarządzania klastrami/centrami obliczeniowymi
- punkt centralny - **VCenter**
  - centralne zarządzanie
  - pojedyncza autoryzacja
- kluczowe funkcjonalności:
  - **vMotion** - do migracji live (bez zatrzymywania VM)
    - kopia pamięci RAM VM do docelowego hosta

    - konieczna jest zgodność procesorów, wspólna pamięć masowa, sieć o wystarczającej przepustowości
    - możliwa automatyzacja
  - **HA - High Availability**
    - maszyna *master* monitoruje stan pozostałych węzłów (*heartbeat*)
    - brak *heartbeat* - węzeł ma stan *failed*, VM restartowana na innym węźle
    - brak łączności sieciowej, ale jest *heartbeat* - stan *isolated/partition*
    - restartowane są tylko maszyny w trybie *failed*
  - **DRS - Distributed Resource Scheduler**
    - równoważenie obciążenia
    - tryb ręczny - tylko propozycje migracji
    - tryb częściowo automatyczny - auto-rozmieszczenie VM po starcie, później propozycje
    - tryb automatyczny - samodzielnie migruje VM
    - możliwość pisania reguł np. “te VM muszą być razem”
- stardard switch vs distributed switch
  - **standard switch SVS** - konfigurowany osobno na każdym węźle, wymaga ręcznego utrzymania spójności ustawień na wszystkich hostach
  - **distributed switch DS** - konfigurowany centralnie w VCenter, ustawienia dystrybuuowane do wszystkich węzłów
- **VMkernel Network**
  - wydzielona sieć wewnątrz ESXi, obsługująca funkcjonalności samego hypervisora
  - obsługuje:
    - sieć zarządczą - dostęp do hosta
    - vMotion - ruch migracji
    - storage
    - VSAN - współdzielenie lokalnej pamięci masowej między węzłami
  - cel - separować sieci od siebie oraz zapewnić odpowiednią przepustowość

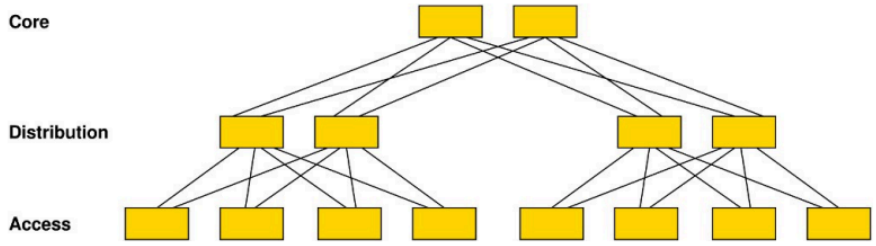
Wykład 11

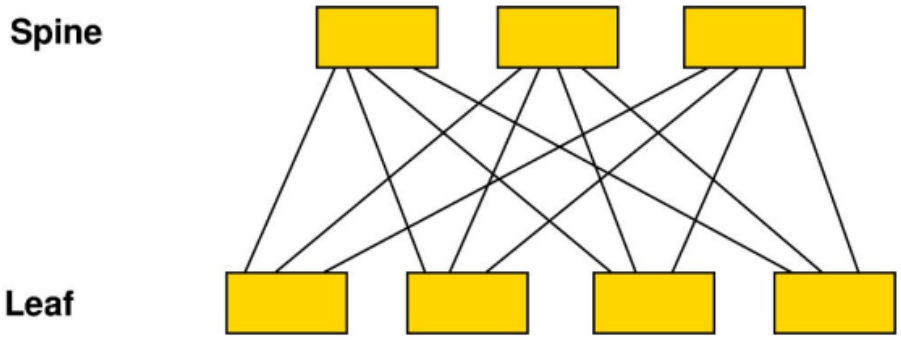
SDN

- ***Software Defined Networking***
- rozdzielenie dwóch funkcjonalności tradycyjnego routera:
  - **Control plane** - logika wyznaczania tras (routing), w SDN wydzielona do osobnego **kontrolera SDN**, który konfiguruje urządzenia sieciowe przez standardowy interfejs
  - **Data plane** - fizyczne przekazywanie pakietów (forwarding)
- na styku tych funkcjonalności działa **lookup** - wybór trasy dla konkretnego pakietu
- sieci **overlay** i **underlay** - SDN często buduje wirtualną sieć nakładkową (**overlay**) na istniejącej infrastrukturze fizycznej (**underlay**), wymaga tunelowania

Topologie sieci overlay

- dla kontekstu:
  - ruch północ-południe - ruch pomiędzy serwerami a internetem
  - ruch wschód-zachód - ruch pomiędzy serwerami wewnątrz centrum danych
- **pojedynczy przełącznik + VLAN (L1)**- proste ale słabo skalowalne
- ***Core-Distribution-Acess*** **(L2)**

- 

  - trójwarstwowa topologia
  - klasyczna hierarchia korporacyjna
  - ruch wschód-zachód jest słabo obsługiwany
  - zapobieganie pętlom, ale blokowanie zapasowych łącz
- ***Leaf-Spine*** **(L3)**
  - 

  - każdy **leaf** (przełącznik dostępowy) jest połączony z każdym **spine** (przełącznik szkieletowy)
  - ruch rozkładany równomiernie
  - dobrze działa dla ruchu wschód-zachód

VMware NSX-T

- implementacja SDN od WMware
- **NXS Manager** - centralny kontroler:
  - dostarcza API i GUI
  - działa w klastrze HA pod jednym adresem IP
  - konfiguruje elementy na hostach ESXi
- **NSX Edge** - lokalny agent wykonujący zadania routingu i firewall dla ruchu wchodzącego/wychodzącego
- hierarchia routerów:
  - **Tier 0 Gateway** - łączność z siecią zewnętrzną (fizycznym routerem)
  - **Tier 1 Gateway** - brama dla segmentów wirtualnych, łączy logiczne przełączniki
- **segmenty** - logiczne przełączniki L2 dla VM
- rozproszony firewall - reguły filtracji działają bezpośrednio na każdym hoście ESXi, możliwa filtracja wschód-zachód

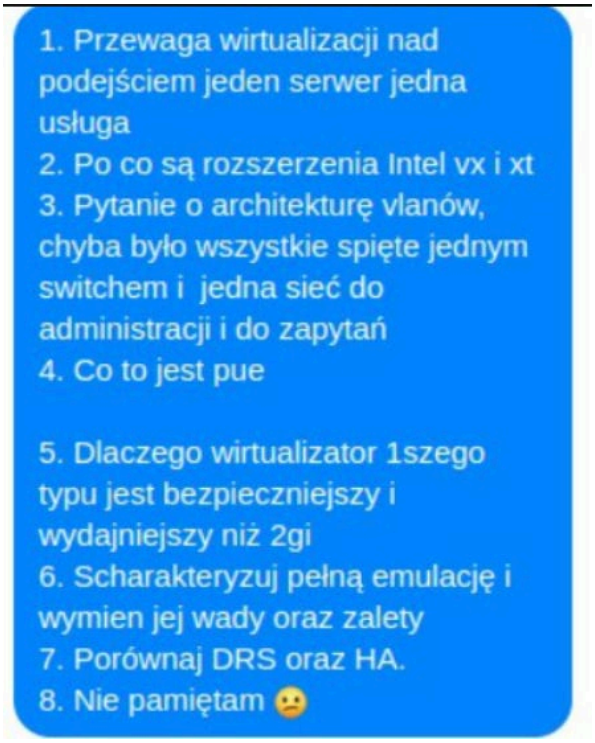
Beton

Otwarte

1. Co to jest thin-provisioning

- Format zapisu wirtualnego dysku na dysku gospodarza. Format ten zajmuje tylko tyle miejsca, ile faktycznie potrzebują i używają VM. Oszczędza to miejsce na maszynie gospodarza, ale dodaje narzut przy alokacji większej ilości miejsca.

2. Dlaczego umieszczenie wszystkich maszyn wirtualnych w jednej sieci (wirtualnym przełączniku, vlanie) jest niebezpieczne? Co należy zrobić, żeby zwiększyć bezpieczeństwo?
   - Wprowadza to *single-point-of-failure* dla całej infrastruktury. Przejęcie jednej maszyny wirtualnej może dać dostęp atakującemu do innych VM, a nawet do maszyny hosta. Aby zwiększyć bezpieczeństwo architektury należałoby ograniczyć możliwość komunikacji pomiędzy VM a hostem, czy innymi VMkami do niezbędnego minimum
3. Jakie są składniki poboru energii przez centrum obliczeniowe? Co można zrobić na etapie projektowania centrum, aby zmniejszyć zużycie energii? Uwaga! Nie chodzi o wykorzystanie mechanizmów programowych.
   - Centrum obliczeniowe pobiera energię do zasilania serwerów, chłodzenia, urządzeń sieciowych, czy systemów monitorowania. Aby zmniejszyć zużycie energii można:
     - ograniczyć zużycie prądu na chłodzenie poprzez wykorzystanie *free-cooling* - zimnego powietrza z zewnątrz gdy temperatura na to pozwala
     - odzyskiwanie ciepła - np. ogrzewanie budynków ciepłem z serwerowni przy wykorzystaniu pompy ciepła
     - zwiększenie efektywności chłodzenia poprzez odpowiednie rozplanowanie przepływu ciepłego i zimnego powietrza
4. Co jest głównym składnikiem poboru mocy procesora (dowolnego układu cyfrowego synchronicznego)
   - Głównym składnikiem poboru mocy jest moc potrzebna do przełączania tranzystorów. Wynosi ona $P_{\text{dyn}} = \alpha C U^2 f$
5. Czy migawka może zostać wykonana w czasie, gdy maszyna jest uruchomiona? Jeśli tak to co można w ten sposób osiągnąć?
   - Migawka może zostać wykonana, gdy maszyna jest uruchomiona, natomiast musi zostać do tego wstrzymana. Umożliwia to zapisanie stanu pamięci RAM i późniejszego jej odtworzenia.
6. Jakie są składniki poboru mocy przez procesor (ogólniej synchroniczny układ cyfrowy)? Dlaczego i jak zużycie energii zależy od napięcia zasilacza i częstotliwości taktowania?
   - $P_{\text{dyn}} = \alpha C U^2 f$ - moc potrzebna do przełączania tranzystorów (we wzorze jest i napięcie i częstotliwość)
   - P_{leak} - mały prąd przepływający przez tranzystor, nawet gdy jest teoretycznie wyłączony, proporcjonalny do napięcia U
   - P_{short} - mały prąd zwracia w chwili przełączania tranzystora, proporcjonalny do częstotliwości f
7. Dlaczego wykorzystanie zewnętrznego firewalla do filtracji ruchu między maszynami wirtualnymi używającymi różnych podsieci (przełączników wirtualnych, vlanów) jest zazwyczaj uważane za bezpieczniejsze niż filtracja na maszynie gospodarza?
   - Filtracja na maszynie gospodarza działa na tym samym OS co hypervisor, co w przypadku przejęcia VM i ucieczki z hypervisora może umożliwić atakującemu modyfikację tych zabezpieczeń. Zewnętrzny firewall jest oddzielną maszyną fizyczną lub VM na innym hoście, posiada własny system operacyjny, przez co przejęcie VM nie daje od razu dostępu do firewalla.

8. 

1. Wiele przewag:
   - efektywniejsze zarządzanie zasobami (jedna usługa na serwerze marnuje jego dostepne zasoby obliczeniowe)
   - niższy koszt wirtualizacji niż kupno jednego sprzętu na usługę
   - łatwiejsze zarządzanie i monitorowanie
2. Intel VT-x - rozszerzenie procesora rozwiązujące problem ring-aliasingu, umożliwia systemom operacyjnym VMek jakby pracować na ringu 0 (z najwyższymi uprawnieniami), gdy VM chce wykonać uprzywilejowaną operację, zgłasza VM Exit, a procesor przekazuje sterowanie do VMM, po obsłużeniu sytuacji VMM zgłasza VM Entry
3. Bez schematu ciężko odpowiedzieć xD
4. *Power Usage Effectiveness* - wskaźnik, ile prądu idzie faktycznie na obliczenia, a ile na obsłużenie infrastruktury:

$$\mathrm{PUE} = \frac{P_{\mathrm{total}}}{P_{\mathrm{IT}}}$$

   - P_{total} - całkowite zużycie energii serwerowni
   - P_{IT} - zużycie energii tylko i wyłącznie na część IT (serwery + sprzęt sieciowy)
5. Wirtualizator 1 stopnia jest szybszy, ponieważ działa bezpośrednio na sprzęcie, a nie na OS gospodarza. Bezpieczniejszy jest w sumie z tego samego powodu, dochodzi kwestia konieczności dbania o bezpieczeństwo i aktualizację OS hosta.
6. Przy pełnej emulacji wirtualizator przechwytuje wszystkie wywołania. Symulacja sprzętu realizowana jest w pełni programowo. Umożliwia to symulowanie zupełnie innych fizycznych maszyn niż maszyna hosta, ogólnie daje największe możliwości. Wadą jest to że jest to ekstremalnie powolny tryb wirtualizacji.
7. Co jest co:
   - HA - *High Availability* - zapewnienie prawidłowego działania systemu poprzez monitorowanie stanów (*heartbeat*) VMek, a następnie ich restartowanie w przypadku awarii

    - DRS - *Distributed Resource Scheduler* - jeden z celów klastrowania, zapewnienie zrównoważonego obciążenia fizycznych zasobów poprzez rozmieszczanie VM na dostępnych zasobach, a następnie ich odpowiednie migracje, umożliwia również migracje z powodu chwilowego wyłączenia maszyny fizycznej, czy pisanie reguł migracji
8. Ta, to zajebiście

Zamknięte

1. Zarządca wirtualny typu 1
    - wymaga do działania systemu operacyjnego na maszynie gospodarza. (Nie)
    - działa bezpośrednio na maszynie gospodarza, więc nie wymaga systemu operacyjnego. (Tak)
    - Wymaga modyfikacji systemu operacyjnego maszyny wirtualnej. (Nie)
2. Zarządca wirtualny pracujący w trybie emulsji (xD, emulacji)
    - może udostępniać inny sprzęt niż obecny na maszynie fizycznej. (Tak)
    - nie może udostępniać innego sprzętu niż obecny na maszynie fizycznej. (Nie)
    - zazwyczaj odznacza się większą wydajnością niż zarządca pracujący w trybie parawirtualizacji. (Nie)
3. Ruter
    - zapewnia globalną łączność dzięki wykorzystaniu adresu IP i protokołów pozwalających na wymianę tras między ruterami. (Tak)
    - nie zapewnia separacji domen kolizyjnych. (Nie)
    - nie zapewnia separacji domen rozgłoszeniowych. (Nie)
4. Migawki o przechowywanie, że wymagają zapisu na maszynie (Chyba chodzi o te tryby zapisu w VirtualBox)
    - Normal - ma migawki
    - Write-through - nie ma
    - Shareable - nie
    - Immutable - koncepcyjnie bez sensu
    - Multiattach - tak
    - read-only - nie
5. O wirtualizację z wsparciem sprzętowym, że wymaga rozszerzenia procesowa Intel VT-x
6. Switch, że zapobiega kolizjom
    - urządzenie w warstwie 2, bridge z wieloma portami, służy do separowania domen kolizyjnych, ale wszystko należy do jednej domeny rozgłoszeniowej
7. Klonowanie vs migawka
    - klonowanie pozwala utworzyć niezależną kopię maszyny, migawka jest uzależniona od swojej VM
    - klonowanie wykonuje pełną kopię informacji, migawka zapisuje tylko różnice
8. Router
    - to urządzenie działające w warstwie sieciowej, przesyła pakiety pomiędzy sieciami przy użyciu protokołu IP, rozdziela domeny rozgłoszeniowe